

Stoquify WhatsApp Execution Action Plan

Date: 2026-07-29 Source strategy: what-next/STOQUIFY_WHATSAPP_INTEGRATION_STRATEGY_2026-07-28.md Scope: Execution-ready blueprint for blending WhatsApp into Stoquify as a controlled communication, evidence, receipt, digest, reminder, and escalation layer.

Executive Action Plan

Stoquify should execute the WhatsApp program as a disciplined communication infrastructure project, not as a chatbot bolt-on.

The first production-safe path is:

1. Establish the WhatsApp policy spine: consent, opt-out, templates, message categories, phone ownership, redaction, RBAC, audit, and cost limits.
2. Build a tenant-safe communication foundation: provider abstraction, contact/consent registry, template registry, outbound message ledger, webhook ingestion, and delivery status tracking.
3. Activate the lowest-risk, highest-trust workflow first: POS receipt delivery by WhatsApp using Stoquify's existing public receipt token and delivery audit patterns.
4. Add daily use loops: owner morning digest and manager action escalation, both redacted and app-link-only for decisions.
5. Add inbound evidence intake: photos and PDFs become draft evidence in Stoquify, never accepted proof until reviewed in the app.
6. Expand into supplier/customer communication, payroll self-service notices, onboarding support, and finally WhatsApp Flows after the communication foundation is proven.

The value thesis is simple: WhatsApp makes Stoquify present in the moments where the business actually operates. Receipts happen at the counter, evidence is captured in the field, suppliers reply from their phones, owners check decisions before opening the dashboard, managers act when nudged, and employees trust payroll notices when they arrive in a familiar channel.

The dashboard remains the source of truth. WhatsApp carries attention, proof, and prompts into Stoquify.

Context That Shapes The Plan

The source report and local inspection show the integration has unusually good fit because Stoquify already has the right anchors:

- services/pos/receipt.service.ts already contains a ReceiptDeliveryProvider interface with sendWhatsAppReceipt, receipt delivery audit, customer phone handling, legal delivery blocking, and digital receipt URLs.
- services/pos/public-receipt-token.ts and services/pos/public-receipt-token-registry.service.ts already support public receipt tokens, expiry, access tracking, revocation, and audit.
- services/daily-habit/daily-habit-digest.service.ts already composes role-aware owner, manager, finance, accountant, stockkeeper, and weekly business digests with evidence grades and redactions.
- services/owner-war-room/owner-war-room.service.ts already builds owner morning briefs, proof-linked actions, stale evidence warnings, payroll forecast redaction, and proof drawer subjects.
- services/manager-action-center/manager-action-center.service.ts already filters action queues by permissions, exposes hidden-by-permission counts, and carries evidence grades and redactions.
- services/reconciliation/payment-reconciliation-notifications.ts already models idempotent notification events with evidence references.
- app/[locale]/(dashboard)/dashboard/settings/notifications/NotificationsSettingsClient.tsx already says email, SMS, WhatsApp, push, digest, and persisted inbox remain adapter-ready.
- what-next/module-surface-inventory.md is report-only and shows module/RBAC mapping is active, with 19 catalog modules, 337 mapped surfaces, 4 missing permissions, and 6 unmapped surfaces as of 2026-07-26.
- The current platform already has audit logs, business events/outbox-style models, workflow assurance alert delivery, agent guardrails, and release gates.

This means the execution plan should extend the existing operating model. It should not create a parallel messaging mini-platform with its own truth, permissions, or approvals.

Current WhatsApp Constraints To Design Around

Current primary-source checks confirm:

- Cloud API is the official platform direction; the on-premises API is deprecated.
- Messages are sent through the phone-number /messages endpoint and return WhatsApp message IDs.
- Webhooks are required for inbound messages, message statuses, read/delivery/failure states, media references, and interactive replies.
- Business-initiated messages must use approved templates outside the customer-service window.
- Businesses can reply with non-template free-form messages inside the 24-hour customer-service window after the user's last message.
- WhatsApp Business policy requires opt-in before contacting users and requires businesses to honor opt-out/block/discontinue requests.
- Pricing is per delivered message, based on recipient market and category: marketing, utility, authentication, and service.
- WhatsApp Flows use interactive messages with flow IDs/tokens and are best treated as a later structured-workflow layer.

Language Locked

- WhatsApp integration: a governed channel layer for sending and receiving WhatsApp messages, not a replacement dashboard, ledger, payroll engine, stock system, or close-certification tool.
- Source of truth: Stoquify database, domain services, permission checks, evidence review, audit logs, and release gates. WhatsApp messages are communication evidence, not business truth by themselves.
- Acknowledgement vs approval: acknowledgement means "the message was seen or someone intends to act"; approval means a controlled business decision performed in Stoquify with RBAC, fresh auth where required, audit, and domain validation.
- Evidence intake: inbound WhatsApp media or structured replies converted into draft evidence records with hashes, sender metadata, and candidate links, then accepted/rejected/redacted by an authorized reviewer inside Stoquify.
- Client engagement loop: a recurring product habit where WhatsApp nudges a user, Stoquify shows the trusted context, the user completes controlled work in-app, and Stoquify reports the outcome back through a safe channel.
- Controlled communication layer: an outbound/inbound messaging system governed by consent, template category, tenant scope, RBAC, redaction, audit, idempotency, rate limits, and cost budgets.

Implementation Spine

State

Stoquify owns all durable state:

- contacts and consent;
- templates and allowed use cases;
- outbound message requests;
- provider message IDs and delivery statuses;
- inbound webhook events;
- draft evidence intake;
- communication preferences;
- audit and cost records.

WhatsApp owns delivery transport and message-channel state only. Stoquify should not infer approval, posting, close certification, payroll validity, or stock truth from WhatsApp delivery/read status.

Data Model And Invariants

Add a communication schema family, tenant-scoped by default:

- CommunicationChannelAccount
- CommunicationContact
- CommunicationConsent
- CommunicationPreference
- CommunicationTemplate
- CommunicationMessage
- CommunicationDeliveryEvent
- CommunicationWebhookEvent
- CommunicationInboundDraft
- EvidenceIntakeDraft

Core invariants:

- A business-initiated outbound message cannot be queued without a valid contact, active consent, allowed template, tenant scope, and channel policy.
- A WhatsApp recipient phone number must be normalized and associated with exactly one active communication contact per tenant unless conflict review is required.
- Raw webhook payloads are immutable and restricted.
- Webhook event processing is idempotent by provider event/message ID.
- Inbound media can create draft evidence only. It cannot post ledgers, approve payroll, receive stock, close periods, certify reports, or activate modules.
- Acknowledgement records are separate from approval records.

Service Boundaries

Create a new services/communication/ boundary for cross-channel communication infrastructure:

- communication-provider.ts
- whatsapp-cloud-api.provider.ts
- communication-contact.service.ts
- communication-consent.service.ts
- communication-template.service.ts
- communication-outbound.service.ts
- communication-webhook.service.ts
- communication-delivery.service.ts
- communication-preference.service.ts

- communication-evidence-intake.service.ts
- communication-cost-control.service.ts

Domain services should call typed communication intents, not raw WhatsApp APIs. Example intents:

- sendReceiptAvailableMessage
- sendOwnerMorningDigestMessage
- sendManagerActionEscalationMessage
- requestEvidenceByWhatsApp
- sendSupplierInvoiceRequestMessage
- sendPayslipAvailableMessage

Contracts

Outbound contract:

- input: tenant ID, source module, source entity, actor/system initiator, template key, recipient reference, locale, variables, sensitivity level, evidence/proof link if any, idempotency key;
- output: queued/sent/skipped/blocked status, reason code, communication message ID, provider message ID when available;
- errors: policy blocked, missing consent, missing template, missing permission, redaction blocked, provider unavailable, rate-limit exceeded.

Inbound contract:

- input: verified webhook envelope;
- output: persisted webhook event, mapped contact if any, created inbound draft if actionable, delivery update if status event, quarantine if unsafe;
- errors: invalid signature, duplicate event, unknown tenant/contact, unsupported media, oversized media, blocked sender, policy violation.

Trust Boundary

Untrusted input enters at the WhatsApp webhook route. It must be verified, persisted, deduplicated, normalized, and routed to draft handlers only.

Trusted business decisions happen in existing Stoquify actions and services after user auth/RBAC/fresh-auth checks.

Sync Model

Use an asynchronous model:

- domain services enqueue communication intents;
- worker/job sends through provider;
- provider response updates message state;
- webhook statuses update delivery state;
- inbound media creates draft evidence;
- UI reads durable communication state.

Start simple with database-backed message states if there is no production-ready queue. Later, route through existing business outbox patterns if they are already hardened.

Failure Handling

- Provider timeout: retry with exponential backoff and idempotency.
- Template blocked/rejected: mark message blocked, surface admin action, do not fallback to free-form if outside the customer window.
- Missing consent: mark skipped/blocked, request consent through in-app or manual workflow.
- Duplicate webhook: record duplicate count, do not duplicate evidence or statuses.
- Unknown sender: create restricted unmatched inbound record, not evidence.
- Media scan failure: quarantine and alert reviewer.
- Cost budget exceeded: pause non-critical sends, continue only critical utility messages if policy allows.
- WhatsApp outage: keep in-app notification and email/SMS fallback if configured.

High-Impact Decisions

Decision	Default	Why It Fits Stoquify	Tradeoff
Provider approach	Direct Cloud API behind a provider interface	The official SDK docs are archived; direct API avoids stale SDK dependency while preserving provider swap later	Requires building/maintaining signing, request, and webhook code
First channel state	Database-backed message ledger	Stoquify already uses Prisma, audit logs, and readiness gates	Not as scalable as a dedicated queue, but safer for MVP
First workflow	POS receipt delivery	Existing receipt service already has WhatsApp stub, token URL, phone handling, and delivery audit	Limited strategic breadth, but fastest trust win
First engagement loop	Owner morning digest and manager escalation	Existing digest/action services already produce redacted, permission-aware summaries	Requires careful template copy and frequency controls
Inbound proof	Draft evidence only	Protects finance, stock, payroll, and close truth	Users may expect instant acceptance; UI must explain review
Approval model	App-only approvals	Preserves RBAC, fresh auth, SoD, and audit	WhatsApp remains less convenient for final decisions
Template governance	Configured template registry	Avoids hardcoded content and category mistakes	Needs admin/review UI and template lifecycle discipline
Metrics	Usage and evidence KPIs from day one	Product value depends on adoption loops, not just message sending	Requires reporting work in Phase 1/2

Phased Execution Roadmap

Phase 0: Policy, Consent, Templates, And Architecture

Objective:

- Freeze the safe communication policy before writing integration code.

Likely files/services touched:

- what-next/
- docs/
- config/
- no runtime source unless adding static policy constants/tests.

Data model changes:

- None yet, but produce final schema proposal for communication tables.

UI changes:

- None yet.

Tests required:

- Static policy test or report gate for prohibited WhatsApp actions.

Risks:

- Product pressure to skip consent/template governance.
- Overpromising "approve from WhatsApp."

Release gate:

- A saved policy artifact that lists allowed, pilot, later, and forbidden WhatsApp workflows.

User value unlocked:

- Clarity. Teams can begin template setup, WABA setup, and copy approval without drifting into unsafe flows.

Exit criteria:

- Consent categories defined: receipts, operational alerts, evidence requests, supplier/customer communication, payroll notices, support, marketing.
- Template starter set reviewed.
- Data retention policy drafted.
- Product owner confirms app-only approvals.

Phase 1: Communication Foundation

Objective:

- Build the shared communication layer once, so every WhatsApp workflow uses the same consent, template, audit, delivery, and webhook controls.

Likely files/services touched:

- prisma/schema.prisma
- services/communication/*
- actions/communication/*
- app/api/communication/whatsapp/webhook/route.ts
- app/[locale]/(dashboard)/dashboard/settings/notifications/*
- components/notifications/*
- lib/notifications/notify.ts
- lib/security/rbac-permissions*
- scripts/module-surface-inventory.js only if new surfaces need classification.

Data model changes:

- Add channel account, contact, consent, preference, template, message, delivery event, webhook event, inbound draft, and evidence intake draft tables.
- Add indexes on tenant, phone hash, provider message ID, idempotency key, template key, status, and source entity.

UI changes:

- Extend notification settings with WhatsApp readiness, channel preferences, consent states, and test-mode delivery.
- Add admin health panel for WABA/phone account status, webhook status, template status, and monthly spend estimate.

Tests required:

- Unit tests for consent blocking, opt-out, idempotency, template validation, phone normalization, provider failure, webhook signature validation, duplicate webhook processing, and tenant isolation.
- Component tests for notification settings WhatsApp states.

Risks:

- Phone numbers can map to multiple tenants.
- Raw webhooks can leak sensitive content.
- Template category mistakes can cause Meta rejection or pricing errors.

Release gate:

- Communication foundation remains report/test mode until webhook verification, opt-out, consent block, duplicate webhook, and provider failure tests pass.

User value unlocked:

- A trusted channel base ready for receipts, digests, evidence, and supplier/customer workflows.

Exit criteria:

- Outbound messages can be queued but are blocked unless template, consent, tenant, and policy pass.
- Inbound webhook events are persisted and deduplicated.
- Admin can see readiness status.
- No domain workflow is activated yet.

Phase 2: POS Receipt Delivery

Objective:

- Deliver secure POS receipt links by WhatsApp with audit and delivery status.

Likely files/services touched:

- services/pos/receipt.service.ts
- services/pos/public-receipt-token.ts
- services/pos/public-receipt-token-registry.service.ts
- actions/pos/*
- components/pos/ProfessionalPOSSystem.tsx
- components/pos/ReceiptTokenControlStrip.tsx
- components/pos/ReceiptTokenHistoryPanel.tsx
- app/api/receipts/[receiptId]/route.ts
- services/communication/*

Data model changes:

- Link CommunicationMessage to SalesOrder and optionally PublicReceiptAccessToken.
- Track receipt delivery channel, provider message ID, status, and failure reason.

UI changes:

- Add WhatsApp receipt option in POS receipt flow.
- Show consent/contact status next to phone number.
- Show delivery state in receipt token history.
- Add "copy WhatsApp link" or click-to-WhatsApp fallback if Cloud API is not configured.

Tests required:

- Receipt send blocks without phone/consent.
- Receipt send blocks when legal delivery policy blocks receipt delivery.
- Receipt send uses tokenized URL and avoids sensitive raw payment details.
- Delivery audit records communication message and receipt/token IDs.
- Provider failure records failed state without breaking sale completion.

Risks:

- Customers expect legal receipt certification from a WhatsApp link.
- Wrong phone number sends receipt to the wrong person.

Release gate:

- Receipt WhatsApp delivery must pass token, audit, consent, and delivery-state tests.

User value unlocked:

- Immediate customer trust at checkout.
- Fewer lost receipts and manual resend requests.
- A visible, easy-to-understand WhatsApp value moment for every client.

Exit criteria:

- Receipt send works in sandbox/test provider.
- Delivery statuses are visible.
- Revoke token still invalidates public access.

Phase 3: Owner Digest And Manager Escalation

Objective:

- Turn WhatsApp into Stoquify's daily operating habit loop without exposing sensitive detail.

Likely files/services touched:

- services/daily-habit/daily-habit-digest.service.ts
- services/owner-war-room/owner-war-room.service.ts
- services/manager-action-center/manager-action-center.service.ts
- services/reconciliation/payment-reconciliation-notifications.ts
- actions/owner-war-room/*
- actions/manager-action-center/*
- app/[locale]/(dashboard)/dashboard/daily-digest/page.tsx
- app/[locale]/(dashboard)/dashboard/owner-war-room/page.tsx
- app/[locale]/(dashboard)/dashboard/manager-action-center/page.tsx
- services/communication/*

Data model changes:

- Add digest source links to CommunicationMessage.
- Add acknowledgement record type for "I will handle" or "Seen", explicitly separate from approval.

UI changes:

- Add WhatsApp delivery preference for owner/manager digests.
- Add "send test digest" in admin settings.
- Add "sent by WhatsApp" and "acknowledged" badges in owner/manager views.

Tests required:

- Digest content redacts payroll/person-level detail.
- User without permission cannot receive hidden action details.
- Acknowledgement does not perform sign-off, payment resolution, stock mutation, or close certification.
- Digest frequency caps work.
- Critical messages are idempotent.

Risks:

- WhatsApp summary can accidentally reveal finance/payroll details.
- Read/acknowledged status can be misunderstood as approval.

Release gate:

- Digest and escalation templates pass redaction review and permission tests.

User value unlocked:

- More frequent owner and manager engagement.
- Faster daily close and reconciliation follow-through.
- Stoquify becomes a daily management habit, not only an end-of-day dashboard.

Exit criteria:

- Owner digest and manager escalation are opt-in, redacted, delivered, audited, and link back to Stoquify.

Phase 4: Inbound Evidence Intake

Objective:

- Convert WhatsApp photos/PDFs into draft evidence that can be reviewed, linked, redacted, and accepted in Stoquify.

Likely files/services touched:

- services/evidence/*
- actions/evidence/proof-trail.actions.ts
- components/evidence/*
- services/communication/communication-evidence-intake.service.ts
- app/api/communication/whatsapp/webhook/route.ts
- services/storage/* or existing upload/evidence storage adapter.

Data model changes:

- Add EvidenceIntakeDraft with sender, message ID, media metadata, hash, candidate subject, quarantine status, review status, reviewer, and accepted proof link if any.

UI changes:

- Add "WhatsApp drafts" tab in evidence drawer or evidence review surface.
- Reviewer can accept, reject, classify, link, redact, quarantine, or request clarification.
- Show source metadata and warning if sender/tenant/contact mapping is uncertain.

Tests required:

- Unknown sender creates restricted unmatched draft, not proof.
- Duplicate media webhook does not duplicate draft.
- Oversized/unsupported media quarantines.
- Accepted evidence requires permission for the target subject.
- Rejected evidence remains audit-visible but excluded from proof trail.

Risks:

- Fraudulent payment proof or delivery note is mistaken as truth.
- Raw media contains private information.
- Malware or unsupported file types enter storage.

Release gate:

- Inbound evidence remains draft-only until review tests, scan/quarantine tests, and redaction tests pass.

User value unlocked:

- Field proof no longer stays trapped in informal chats.
- Close, AP, stock, and reconciliation blockers can be cleared faster.

Exit criteria:

- WhatsApp media can become reviewed Stoquify evidence, but never automatic truth.

Phase 5: Supplier And Customer Communication

Objective:

- Capture supplier/customer communication around POs, invoices, receipts, payment reminders, and support in controlled records.

Likely files/services touched:

- services/supplier/*
- actions/suppliers/*
- services/customer/*
- actions/customers/*
- components/suppliers/*
- components/customers/*
- app/[locale]/(dashboard)/dashboard/purchases/suppliers/*
- app/[locale]/(dashboard)/dashboard/customers/*
- services/communication/*

Data model changes:

- Link CommunicationContact to supplier/customer records.
- Link message threads to PO, invoice, customer ledger, or support subject.

UI changes:

- Add WhatsApp contact status on customer/supplier profile.
- Add click-to-WhatsApp and Cloud API send actions where consent exists.
- Show communication timeline filtered by permission.

Tests required:

- Supplier/customer messages require contact consent and correct module permissions.
- Payment reminder template category is selected correctly.
- Supplier invoice replies remain draft evidence until AP review.

Risks:

- Collections messages become too aggressive or are miscategorized as utility.
- Supplier documents are treated as liabilities before AP review.

Release gate:

- Start with supplier invoice request and customer receipt support, not marketing.

User value unlocked:

- Suppliers respond faster.
- Customers get better proof and follow-up.
- Informal communication becomes governed operating evidence.

Phase 6: Payroll Self-Service Notices

Objective:

- Notify employees that payslips or payroll tasks are available without exposing sensitive payroll values in WhatsApp.

Likely files/services touched:

- `services/payroll/payslip-self-service.service.ts`
- `actions/payroll/payroll-payslip-self-service.actions.ts`
- `components/payroll/PayrollPayslipSelfService.tsx`
- `services/payroll/payroll-privacy.service.ts` if present/adjacent
- `services/communication/*`

Data model changes:

- Link employee communication contact to `PayrollEmployee`/user identity.
- Add payroll-specific consent/preference category.

UI changes:

- Employee self-service notification preferences.
- Payroll admin delivery status for notices only.

Tests required:

- Message contains no salary, bank, tax, statutory, or person-level sensitive detail.
- Employee phone must be verified/linked before delivery.
- Self-service link requires login/RBAC.
- Opt-out is honored.

Risks:

- Payroll confidentiality leakage.
- Old phone number receives notice.

Release gate:

- Payroll WhatsApp messages are "availability notices" only.

User value unlocked:

- Employee trust improves.
- HR/payroll teams reduce manual payslip follow-up.

Phase 7: WhatsApp Flows And Advanced Adoption Loops

Objective:

- Use structured in-chat forms only after messaging, consent, webhook, and evidence review foundations are stable.

Likely files/services touched:

- `services/communication/whatsapp-flows.service.ts`
- `services/inventory/*`
- `services/purchasing/*`
- `services/hris/*`
- `components/*` for review surfaces.

Data model changes:

- Add flow definition registry and flow run/session records.

UI changes:

- Admin flow registry.
- Review queues for stock count, supplier onboarding, field incident, and leave/attendance flow submissions.

Tests required:

- Flow token idempotency.
- Flow response cannot mutate stock/payroll/finance directly.
- Flow submission links to app review.

Risks:

- Structured WhatsApp forms become shadow workflows.
- Users expect completion in WhatsApp to equal approval.

Release gate:

- Every Flow submission lands in a review queue first.

User value unlocked:

- Low-literacy users can submit structured information without learning complex screens.
- Field operations can become digital without forcing everyone into the full app first.

MVP Definition

MVP name: WhatsApp Evidence and Operations Notification Layer.

MVP includes:

1. POS receipt delivery by WhatsApp link.
2. Owner morning digest with redacted metrics and Stoquify links.
3. Manager daily-close/reconciliation escalation with acknowledgement only.
4. Inbound evidence photo/PDF intake as draft evidence.

MVP excludes:

- payroll approvals;
- stock adjustments;
- payment posting;
- close certification;
- statutory filing;
- module entitlement changes;
- marketing campaigns;
- WhatsApp Flows;
- AI chatbot automation beyond templated support routing.

MVP release principle:

- Users may receive, acknowledge, and submit draft proof through WhatsApp. They must open Stoquify for approval, posting, sign-off, correction, review, or certification.

Workflow Blending Plan

POS Receipt Screen

- Add WhatsApp as a receipt channel next to print/email/SMS.
- Show customer phone, consent status, delivery status, and token expiry.
- If Cloud API is not configured, offer click-to-WhatsApp fallback with prefilled receipt link.

Customer Profile

- Show WhatsApp opt-in category, last delivery, last inbound, receipt support status, and communication history.
- Keep collections/payment reminders behind finance/customer permission and category policy.

Supplier Profile

- Show WhatsApp contact verification and consent.
- Add PO dispatch and invoice request actions.
- Inbound supplier docs land in AP/evidence review.

Evidence Drawer

- Add a "WhatsApp drafts" section.
- Show source phone/contact, message ID, timestamp, hash, media type, proposed subject, scan state, and reviewer decision.

Owner War Room

- Show owner digest delivery history and acknowledgement state.
- Keep proof-linked actions as app deep links.
- Do not expose raw payroll/person-level values in WhatsApp summaries.

Manager Action Center

- Add escalation send and acknowledgement status.
- Distinguish "acknowledged by WhatsApp" from "reviewed/signed in Stoquify."
- Use hidden-by-permission and redaction state to suppress unsafe details.

Payroll Self-Service

- Add WhatsApp notice preference for employees.
- Send only "payslip available" or "action needed" notices with secure app links.
- Keep payslip content inside Stoquify.

Notification Settings

- Promote the current adapter-ready channel text into real channel controls.
- Add WhatsApp channel health, templates, consent categories, quiet hours, digest frequency, and spend limits.

Admin Configuration

- Add WABA/phone number ID readiness.
- Show webhook verification state, last webhook event, template approval state, provider errors, opt-out volume, and delivery/cost trend.

Adoption And Value Strategy

Daily Digest Habit Loop

- Trigger: owner/manager receives safe morning summary.
- Action: opens Stoquify only for the real decision.
- Reward: sees what is blocked, stale, risky, or ready.
- Value: Stoquify becomes a daily operating rhythm.

Receipt Trust Loop

- Trigger: customer receives receipt link instantly.
- Action: customer opens or saves receipt.
- Reward: business looks professional and trustworthy.

- Value: POS use feels visibly better to cashiers and customers.

Evidence Capture Loop

- Trigger: Stoquify asks for missing proof.
- Action: user replies with photo/PDF.
- Reward: blocker moves into review.
- Value: fewer close, AP, payroll, stock, and reconciliation delays.

Supplier Response Loop

- Trigger: supplier receives PO/invoice request where they already communicate.
- Action: supplier replies with document or confirmation.
- Reward: AP/purchasing has a controlled trail.
- Value: informal supplier reality becomes formal evidence.

Manager Accountability Loop

- Trigger: manager receives daily-close/reconciliation escalation.
- Action: acknowledges and opens app.
- Reward: action queue clears faster.
- Value: less invisible operational drift.

Employee Payroll Trust Loop

- Trigger: employee receives payslip availability notice.
- Action: opens self-service.
- Reward: timely payroll visibility without exposing data in chat.
- Value: fewer payroll questions and stronger employee trust.

Onboarding/Support Loop

- Trigger: user gets short WhatsApp guidance.
- Action: opens relevant Stoquify surface or asks support.
- Reward: faster adoption.
- Value: clients stay active after onboarding.

Security And Compliance Controls

- Consent: category-specific opt-in is required before business-initiated messages.
- Opt-out: STOP/block/discontinue requests suppress future messages in that category or globally according to policy.
- Phone ownership: verify phone before sensitive notices; show last-four confirmation before sending.

- Tenant isolation: every contact, consent, message, webhook, and draft is tenant-scoped.
- RBAC: outbound domain messages require sender/system authorization and recipient eligibility.
- Module entitlement: WhatsApp must not expose workflows for modules the tenant does not have.
- Redaction: message templates use redacted summaries by default; sensitive values remain in app.
- Fresh auth: high-value actions remain app-only and fresh-auth protected where existing policy requires it.
- Webhook verification: verify challenge and signatures before processing.
- Idempotency: dedupe outbound by idempotency key and inbound by provider message/event ID.
- Audit logs: record who/what triggered message, template, recipient, source entity, delivery state, and webhook events.
- Rate limits: cap sends per tenant, recipient, category, and source event.
- Cost controls: tenant-level monthly budget, category budgets, non-critical pause threshold, admin visibility.
- Safe errors: no raw provider secrets, phone numbers, or message bodies in broad logs.
- Provider secrets: store tokens securely and rotate.
- Fallback: in-app notification remains available when WhatsApp fails.

Engineering Backlog

Database / Schema

- Add communication account/contact/consent/preference/template/message/delivery/webhook/inbound/evidence draft tables.
- Add indexes for tenant, phone hash, provider message ID, template key, source entity, status, and idempotency.
- Add enums for channel, consent status, message status, delivery status, template category, inbound draft status, evidence intake status.
- Add relations to SalesOrder, PublicReceiptAccessToken, Customer, Supplier, PayrollEmployee, and evidence subjects where appropriate.

Services

- Add services/communication.
- Add provider interface and Cloud API provider.
- Add template registry and variable validator.
- Add consent/preference resolver.
- Add outbound policy resolver.
- Add cost-control resolver.
- Add delivery status updater.
- Add evidence intake draft service.

API / Webhooks

- Add WhatsApp webhook route.
- Add webhook verification and signature check.

- Add status-event handler.
- Add inbound-message handler.
- Add media metadata/download handler.
- Add admin test-send route/action where safe.

Queues / Jobs

- Add database-backed outbound processing first.
- Add retry scheduler.
- Add stale pending message cleanup.
- Add delivery reconciliation job.
- Add evidence media quarantine/scan job.
- Later, consolidate with existing business outbox if appropriate.

UI Surfaces

- Extend notification settings.
- Add communication admin health panel.
- Add POS WhatsApp receipt controls.
- Add receipt delivery history.
- Add owner/manager delivery badges.
- Add evidence intake draft review.
- Add customer/supplier contact consent panels.
- Add payroll self-service WhatsApp preference.

Tests

- `services/communication/__tests__/*`
- receipt delivery tests around WhatsApp provider/consent/audit.
- webhook verification and idempotency tests.
- evidence intake draft tests.
- owner/manager redaction and permission tests.
- notification settings component tests.
- module surface inventory refresh after new actions/pages.

Admin Operations

- WABA setup checklist.
- Template approval tracker.

- Provider token rotation runbook.
- Webhook replay/runbook.
- Cost monitoring playbook.
- Opt-out handling runbook.
- Incident response for wrong-recipient send.

Documentation / Runbooks

- docs/whatsapp/architecture.md
- docs/whatsapp/template-governance.md
- docs/whatsapp/consent-policy.md
- docs/whatsapp/incident-runbook.md
- what-next/ phase readiness reports.

Success Metrics

Metric	Target Direction	Why It Matters
WhatsApp opt-in rate	Up	Shows client/channel acceptance
Receipt delivery success rate	95%+	Proves operational reliability
Receipt resend support requests	Down	Measures concrete client benefit
Owner digest open-through rate	Up	Measures executive engagement
Manager action completion time	Down	Measures accountability loop
Evidence completion time	Down	Measures close/AP/reconciliation acceleration
Draft evidence acceptance rate	Healthy, not 100%	Shows review is active, not rubber-stamped
Reconciliation blocker aging	Down	Measures finance impact
Supplier response time	Down	Measures purchasing/AP value
Payroll notice completion rate	Up	Measures employee self-service adoption
Opt-out/block rate	Low/stable	Measures message quality and fatigue
Monthly message cost per active tenant	Controlled	Measures scalable economics
Daily active business users	Up	Measures usage lift
Support ticket reduction	Down	Measures onboarding/support value

Non-Goals And Hard Prohibitions

Do not build:

- direct payroll approval over WhatsApp;
- direct stock adjustment over WhatsApp;
- direct close certification over WhatsApp;
- statutory filing over WhatsApp;

- payment posting over WhatsApp;
- payment reconciliation sign-off over WhatsApp;
- module entitlement activation/deactivation over WhatsApp;
- supplier payable creation from WhatsApp media without AP review;
- automatic evidence acceptance from WhatsApp media;
- marketing campaigns in the MVP;
- a standalone WhatsApp dashboard separate from Stoquify's existing surfaces.

Do not send in WhatsApp message text:

- salary amounts;
- bank account details;
- payment card details;
- sensitive statutory identifiers;
- raw ledger details;
- private payroll/person-level values;
- legal certification claims unless the relevant policy gate explicitly allows them.

Quality Risks Handled

- Data integrity: all outbound and inbound events use idempotency keys and immutable provider event IDs.
- API contract: provider is isolated behind a typed interface, so Stoquify can move between direct Cloud API and BSP later.
- State flow: domain services emit communication intents; communication services handle delivery; domain truth remains unchanged.
- UI state: every WhatsApp surface must show configured, not configured, pending, sent, delivered, read, failed, blocked, opted out, and permission denied states.
- Auth/security: recipient eligibility and sender permissions are resolved before queuing sensitive messages.
- Observability: message ledger, delivery events, webhook events, cost counters, and admin health panel are first-class.
- Performance: use queued sends and batched digest generation; avoid synchronous provider calls inside checkout or close flows.
- Accessibility/i18n: templates and settings need locale-aware copy, including English/French from the start.
- Testing: every phase has focused service, component, action, and module surface tests.
- Rollout: feature-flag by tenant and workflow, starting with test/sandbox delivery.
- Maintainability: WhatsApp stays under services/communication, with domain-specific adapters as callers, not owners.

Failure Modes Considered

- Bad phone number: block send, request verification, show contact correction action.
- Duplicate webhook: ignore duplicate business effect, record duplicate observation.
- Provider outage: keep message pending/failed, retry, and preserve in-app notification.

- Template rejected: block sends using that template and surface admin remediation.
- User opts out: update suppression immediately and stop matching category sends.
- User replies "approve": record inbound message only; provide app link if appropriate.
- Unknown sender submits evidence: create restricted unmatched draft, not proof.
- Media download fails: retry while URL is valid, then mark expired and request re-upload.
- Receipt token revoked after message sent: public receipt URL correctly fails/requires new token.
- Manager lacks permission: do not include hidden action detail in digest.
- Owner digest contains payroll aggregate: keep aggregate only and redact person-level data.
- Cost spike: non-critical sends pause automatically by tenant/category budget.

Edge Cases Considered

- Same phone number used by two tenants.
- Employee changes phone after opt-in.
- Customer has phone but no WhatsApp account.
- Message delivered but not read.
- Read webhook arrives before sent/delivered state.
- Customer replies outside expected thread.
- Media arrives without caption/context.
- Flow response arrives after underlying task was already completed in app.
- Unicode names and localized template variables.
- Very long supplier/customer names in templates.
- Quiet-hours delivery windows.
- Tenant has WhatsApp disabled but click-to-WhatsApp fallback is allowed.

Six-Month Future Scope Check

What bends well:

- Provider abstraction can support direct Cloud API, BSP, or multi-tenant embedded signup.
- Template registry can support Flows later.
- Contact/consent registry can support email/SMS/push without duplicating policy.
- Evidence intake can support OCR/classification later.
- Digest delivery can become multi-channel.

What breaks if skipped now:

- No consent registry means unsafe scaling.

- No message ledger means no audit, no delivery evidence, no support diagnostics.
- No template governance means rejections, cost surprises, and policy risk.
- No draft-only evidence boundary means WhatsApp becomes an unsafe source of truth.
- No acknowledgement/approval separation means users will treat a reply as an authorization.

Verification Commands

Recommended focused checks by phase:

```
npm run prisma:validate
npm run typecheck
npm test -- services/communication
npm test -- services/pos/__tests__/receipt*
npm test -- services/evidence/__tests__
npm test -- services/reconciliation/__tests__
npm test -- components/notifications
npm test -- components/pos/__tests__
npm test -- components/manager-action-center/__tests__
npm run module:surface:inventory
npm run policy:gates
```

Manual smoke checks:

- webhook verification challenge succeeds;
- duplicate webhook is idempotent;
- opted-out contact is blocked;
- WhatsApp receipt delivery writes message audit;
- delivery status updates from webhook;
- owner digest contains only redacted safe summary;
- manager acknowledgement does not approve anything;
- inbound media creates draft evidence only;
- payroll notice contains no salary or statutory-sensitive detail;
- WhatsApp disabled tenant sees safe fallback states.

Final Execution Recommendation

Safest first build:

- Phase 0 plus Phase 1 foundation, then Phase 2 POS receipt delivery. This turns an already adapter-shaped receipt surface into a visible, low-risk WhatsApp win.

Highest-value second build:

- Owner morning digest and manager escalation, because these create daily usage loops and turn Stoquify into a management habit.

Biggest architectural risk:

- Treating WhatsApp as an action surface instead of a communication/evidence surface. If WhatsApp can approve, post, certify, or mutate business truth too early, it weakens Stoquify's strongest controls.

Product-owner decision required before implementation:

- Confirm that the first production release will allow WhatsApp only for delivery, acknowledgement, support, and draft evidence intake, while every approval, posting, stock mutation, payroll effect, close sign-off, statutory action, and module entitlement change remains inside Stoquify.

Blueprint ready.

Sources

Local sources:

- what-next/STOQUIFY_WHATSAPP_INTEGRATION_STRATEGY_2026-07-28.md
- what-next/module-surface-inventory.md
- services/pos/receipt.service.ts
- services/pos/public-receipt-token.ts
- services/pos/public-receipt-token-registry.service.ts
- services/daily-habit/daily-habit-digest.service.ts
- services/owner-war-room/owner-war-room.service.ts
- services/manager-action-center/manager-action-center.service.ts
- services/reconciliation/payment-reconciliation-notifications.ts
- app/[locale]/(dashboard)/dashboard/settings/notifications/NotificationsSettingsClient.tsx
- components/notifications/*
- actions/evidence/proof-trail.actions.ts
- actions/payroll/*
- actions/pos/*
- prisma/schema.prisma
- package.json

Primary WhatsApp/Meta sources:

- WhatsApp Business Platform official Postman overview: <https://www.postman.com/meta/whatsapp-business-platform/overview>
- WhatsApp Business Platform pricing: <https://whatsappbusiness.com/products/platform-pricing/>
- WhatsApp Business policy: <https://whatsappbusiness.com/policy/>
- WhatsApp Flows send request example: <https://www.postman.com/meta/whatsapp-business-platform/request/ftdtf2c/send-published-flow-by-id>
- WhatsApp Business Platform Node.js SDK documentation, archived: <https://whatsapp.github.io/WhatsApp-Nodejs-SDK/>